



Qiskit



python

Qiskit'e Giriş & Kuantum Mantık Kapıları

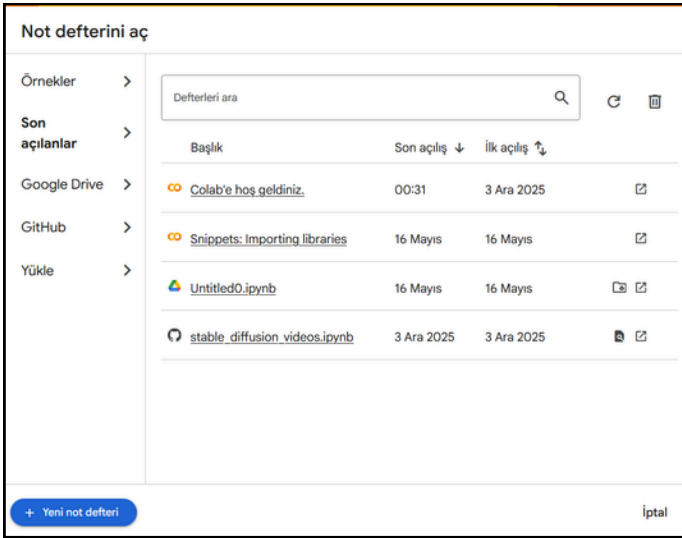
-Mehmet Emin Deniz-

Çalıştırma Ortamının Kurulması:

Google Colab:

Bilgisayarına hiçbir şey yüklemeden, Google hesabıyla doğrudan web tarayıcın üzerinden çalıştırabilirsin.

colab.research.google.com 1- Adrese Git

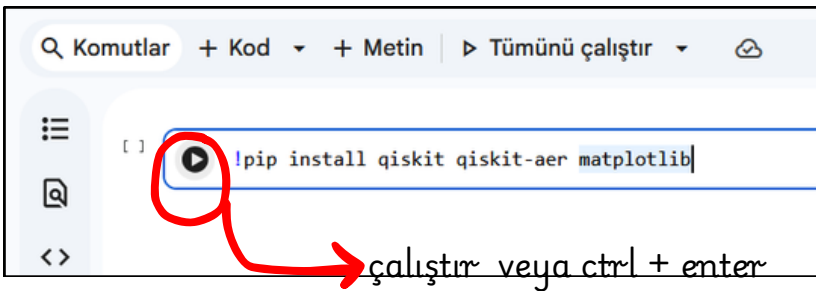


2- "Yeni Not Defteri" (New Notebook) butonuna tıkla.

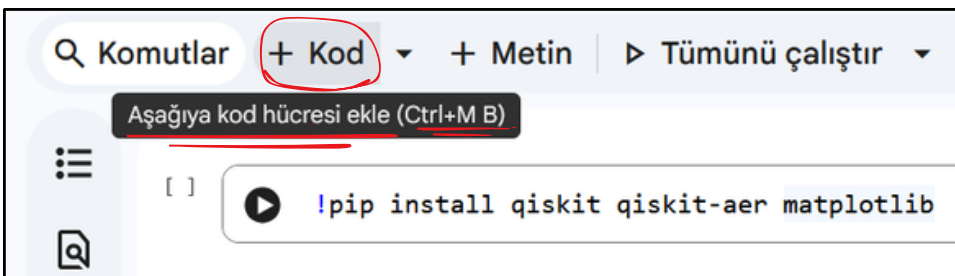
3- İlk hücreye şu indirme komutunu yaz ve çalıştır

```
!pip install qiskit qiskit-aer matplotlib
```

Bash



4- Yeni bir kod hücresi aç



kodlar
çalıştırılmaya
hazır

-QİSKİT'e GİRİŞ-

1. Temel Kavramlar & Mimarisi

Kubit (Qubit): Klasik bitler sadece 0 veya 1 değerini alırken; kubit, ölçüm yapılana kadar bu iki durumun lineer birleşimi (süperpozisyon) halinde bulunur.

Klasik Bit: Kuantum çöktükten sonra çıkan nihai ölçüm sonucunu (0 veya 1) saklamak için kullanılan standart bilgisayar hafızasıdır.

Devre (Quantum Circuit): Kubit hatlarının yan yana dizildiği ve üzerlerine sırayla kapıların işlendiği akış şemasıdır.

Süperpozisyon (Superposition): Kubitin aynı anda birden fazla olasılığı barındırma durumudur (H kapısı ile yapılır).

Dolaşıklık (Entanglement): İki farklı kubitin durumunun birbirine bağımlı hale gelmesidir. Birini ölçtüğün an, diğerinin durumu da anında belirlenir (CNOT kapısı ile yapılır).

Qiskit, klasik Python kodlarıyla bir kuantum devresi (quantum circuit) tasarlama ve bunu simüle etme aracıdır.

Kuantum programlamayı bir müzik bestelemeye benzetebilirsin:

Çizgiler (Teller): Her bir çizgi bir Kubit'tir (Kuantum biti).

Notalar: Kubitlerin üzerine koyduğun Kapılardır (Hadamard, CNOT vb.).

Beste (Devre): Kapıları yan yana dizerek oluşturduğun Kuantum Devresidir.

Dinlemek (Ölçüm): Şarkıyı çalip çıkacak sesi duymaktır. (Kuantumda buna Ölçüm / Measurement denir).

Qiskit'in Mantığı: Sadece 4 Adım:

Qiskit'te yazacağın en basit programdan en karmaşık algoritmaya kadar her kod şu 4 adımdan oluşur:

- Kubitleri Tanımla: Kaç tane kuantum bitin olacak? (Örn: 2 kubit).
- Kapıları Ekle: Kubitlere ne işlem yapılacağını söyle (Süperpozisyon, dolaşıklık vb.).
- Ölçüm Yap: Kuantum durumunu klasik 0 ve 1 sonuçlarına dönüştür.
- Çalıştır ve Sonucu Gör: Kodu bir bilgisayar simülatöründe koştur.

Kapı	Kod	Ne İşe Yarar?
Hadamard (H)	devre.h(0)	Kubiti süperpozisyona sokar (%50 0, %50 1 olasılığı yaratır).
Pauli-X	devre.x(0)	Kuantum NOT kapısıdır. 0 olan kubiti 1, 1 olanı 0 yapar.
CNOT	devre.cx(0, 1)	İki kubiti birbirine bağlar (Dolaşıklık yaratır). 0. kubit 1 ise, 1. kubiti değiştirir.

Kuantum Mantık Kapıları:

Klasik bilgisayarlardaki AND, OR, NOT kapıları elektrik voltajını (0V veya 5V) değiştirirken, Kuantum Mantık Kapıları kubitlerin kuantum durumlarını yönlendiren mantıksal ve fiziksel işlemlerdir.

Matematiksel olarak her kuantum kapısı unitery (birimsel) bir matristir. Yani kuantum kapıları geri döndürülebilir (reversible): Kapıdan geçtikten sonra hangi durumdan geldiğinizi matematiksel olarak kesin olarak bulabilirsiniz (Ölçüm anı hariç).

1. Tek Kubitlik Temel Kapılar

1. Pauli-X Kapısı (devre.x()) — Kuantum NOT Kapısı
Klasik NOT kapısının kuantumdaki birebir karşılığıdır.
Kubitin durumunu tamamen tersine çevirir.

- $|0\rangle \rightarrow |1\rangle$
- $|1\rangle \rightarrow |0\rangle$

Kullanımı: Bir kubit 0 durumundan 1 durumuna başlatmak için algoritmanın en başında kullanılır.

2. Hadamard Kapısı (devre.h()) — Süperpozisyon Kapısı
Kuantum programlamanın en kritik kapısıdır. Kesin bir durumda olan (0 veya 1) kubit süperpozisyona sokar. Yani kubit aynı anda hem 0 hem 1 olma olasılığı kazanır (yazı-tura parasını havaya atmak gibi).

- $|0\rangle \rightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}}$ (%50 0, %50 1)
- $|1\rangle \rightarrow \frac{|0\rangle - |1\rangle}{\sqrt{2}}$

Kullanımı: Bir kuantum algoritmasında tüm olası çözümleri aynı anda hesaplamaya başlamak için her zaman ilk olarak H kapısı uygulanır.

3. Pauli-Y ve Pauli-Z Kapıları (devre.y(), devre.z())

Kubitleri karmaşık düzlemde veya kuantum fazında yönlendirmek için kullanılır.

- Z Kapısı (Phase-Flip): Kubitin genliğini değiştirmeden fazını (işaretini) değiştirir.

$|0\rangle$ durumuna dokunmaz, $|1\rangle$ durumunu $-|1\rangle$ yapar.

- Y Kapısı: Hem bit değişimini (X) hem de faz değişimini (Z) aynı anda yapar.

4. Faz Kapıları: S ve T Kapıları (devre.s(), devre.t())

- S Kapısı: Kubit üzerinde 90 derece ($\pi/2$) faz kayması oluşturur. (Z kapısının kareköküdür).
- T Kapısı: Kubit üzerinde 45 derece ($\pi/4$) faz kayması oluşturur. Kuantum hesaplamada çok yüksek hassasiyetli açılar elde etmek için kullanılır.

2. Çok Kubitlik (Etkileşimli) Kapılar

Kuantum bilgisayarlarını güçlü kılan, kubitlerin birbiriyle etkileşime girip Dolaşıklık (Entanglement) oluşturmasını sağlayan kapılardır.

1. CNOT / CX Kapısı (devre.cx(kontrol, hedef))

— Kontrollü NOT

İki kubit ile çalışır: Bir Kontrol Kubit ve bir Hedef Kubit.

- Mantık: Eğer kontrol kubit 1 ise, hedef kubitine NOT (X) kapısı uygula. Kontrol kubit 0 ise hedef kubit dokunma.

Kontrol Kubit	Hedef Kubit (Girdi)	Hedef Kubit (Çıktı)
0	0	0
0	1	1
1	0	1 (Değişti)
1	1	0 (Değişti)

- Önemli: H kapısı ile CNOT kapısı arka arkaya kullanıldığında ortaya ünlü Bell Durumu (Dolaşıklık) çıkar.

2. CZ Kapısı (devre.cz()) — Kontrollü Z Kapısı

Eğer kontrol kubit 1 ise, hedef kubitine Z kapısı uygula (fazını değiştir).

İki kubitin yer değiştirmesi sonucu değiştirmez (simetriktir).

3. Toffoli / CCNOT Kapısı (devre.ccx(k1, k2, hedef))

— 3 Kubitlik Kapı

==> İki kontrol kubit ve bir hedef kubit vardır.

- Mantık: Yalnızca her iki kontrol kubit de 1 ise hedef kubitini tersine çevirir. Klasik bilgisayarlardaki AND mantığını kuantum dünyasında çalıştırmak için kullanılır.

4. SWAP Kapısı (devre.swap(q1, q2))

İki kubitin kuantum durumlarını birebir birbiriyle takas eder.

1'deki kuantum durumu q_2'ye, q_2'deki durum q_1'e geçer.

3. Kapıların Qiskit Üzerindeki Uygulama Özeti

```

from qiskit import QuantumCircuit

# 2 Kubitlik bir devre
devre = QuantumCircuit(2)

# --- Tek Kubit Kapıları ---
devre.x(0)      # 0. kubit 1 yap
devre.h(0)      # 0. kubit süperpozisyona sok
devre.z(0)      # 0. kubite faz kayması uygula

# --- Çok Kubitli Kapılar ---
devre.cx(0, 1)
# 0. kubit kontrol, 1. kubit hedef olacak şekilde CNOT
devre.swap(0, 1)
# 0 ve 1. kubitlerin durumlarını takas et

```

Özet Tablo (Hangi Kapı Ne İçin Kullanılır?)

Kapı	Kod	Temel Görevi
X	devre.x()	Bit değiştirme (0 \1).
H	devre.h()	Olasılık yaratma (Süperpozisyon).
Z	devre.z()	İşaret/Faz değiştirme.
CX (CNOT)	devre.cx()	İki kubit birbire bağlama (Dolaşıklık).
CCX (Toffoli)	devre.ccx()	Mantıksal AND işlemi yapma.

Örnek: Qiskit ile süperpozisyon (yazı-tura gibi %50 ihtimalle 0 veya 1 gelme durumu) yaratan en temel kod.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

# 1. ADIM: 1 Kubitlik ve 1 Klasik Bitlik bir devre oluştur
qc = QuantumCircuit(1, 1)

# 2. ADIM: H (Hadamard) kapısını ekle.
# Bu kapı kubitini "süperpozisyona" sokar (Parayı havaya atmak
# gibi).
qc.h(0)

# 3. ADIM: Ölçüm yap (Havada dönen parayı yakalayıp masaya
# koy)
# 0 numaralı kubitini ölç, sonucu 0 numaralı klasik bite yaz.
qc.measure(0, 0)

# 4. ADIM: Simülatörde 1000 kez çalıştır
simulator = AerSimulator()
job = simulator.run(qc, shots=1000)
result = job.result()

# Sonuçları ekrana bas
counts = result.get_counts()
print(counts)
```

Bu Kodun Çıktısı Ne Olur?

{'0': 502, '1': 498} gibi bir sonuç alırsın. Yani 1000 denemenin yaklaşık 500'ünde 0, 500'ünde 1 gelir. Tam bir kuantum yazı-turası!

İlk Kuantum Kodunun Satır Satır Analizi:
İşte en temel Qiskit kodumuz (yazı tura sorusu)
ve her bir satırın Türkçe karşılığı:

```
● ● ●  
# =====  
# ADIM 1: GEREKLİ KÜTÜPHANELERİ ÇAĞIRMA  
# =====  
from qiskit import QuantumCircuit  
from qiskit_aer import AerSimulator  
  
# =====  
# ADIM 2: BOŞ BİR KUANTUM DEVRESİ KURMA  
# =====  
devre = QuantumCircuit(1, 1)  
  
# =====  
# ADIM 3: KUBİTE İŞLEM YAPMA (KAPI EKLEME)  
# =====  
devre.h(0)  
  
# =====  
# ADIM 4: ÖLÇÜM (SONUCU BAKMA/KAYDETME)  
# =====  
devre.measure(0, 0)  
  
# =====  
# ADIM 5: SİMÜLATÖRÜ HAZIRLAMA VE ÇALIŞTIRMA  
# =====  
simulator = AerSimulator()  
islem = simulator.run(devre, shots=1000)  
sonuc = islem.result()  
  
# =====  
# ADIM 6: SONUÇLARI EKRANA YAZDIRMA  
# =====  
print(sonuc.get_counts())
```

Kodların Satır Satır Detaylı Açıklaması:

1. Kütüphaneleri İçer Aktarma (Import)



```
# ADIM 1: GEREKLİ KÜTÜPHANELERİ ÇAĞIRMA  
from qiskit import QuantumCircuit  
from qiskit_aer import AerSimulator
```

- **QuantumCircuit:** Bizim kuantum tuvalimizdir. Kubitleri dizip üzerlerine kapılar koyacağımız devre mimarisini sağlar.
- **AerSimulator:** Gerçek bir kuantum bilgisayarımız olmadığı için, kuantum davranışlarını kendi bilgisayarımızda taklit eden sanal kuantum işlemcisidir.
- Bu bir fonksiyon mu, nesne mi?
QuantumCircuit bir Sınıftır (Class).
Sen QuantumCircuit(1, 1) yazdığında, bu sınıftan yeni bir nesne (object) türetirsin.
Yazılımdaki benzetmesi şudur: QuantumCircuit kalıbıdır, devre ise o kalıptan çıkardığın boş bir defter/tuvaldir.

2. Devreyi Oluşturma



ADIM 2: BOŞ BİR KUANTUM DEVRESİ KURMA

```
devre = QuantumCircuit(1, 1)
```

Parantez içindeki sayılar sırasıyla şunlardır:

QuantumCircuit(Kubit Sayısı, Klasik Bit Sayısı)

- 1. Parametre (1): "Bana 1 tane Kubit ver."
(Kuantum işlemi burada yapacağız).
Süperpozisyonu, kapıları bu çizgide çalıştırırız.
İndeksi 0'dan başlar.
- 2. Parametre (1): "Bana 1 tane Klasik Bit ver."
(Kuantum ölçüm sonucunu 0 veya 1 olarak saklamak için normal bir bilgisayar hafızası).
Ölçüm yaptığımızda kuantum çökebilir ve elimizde sadece klasik 0 veya 1 kalır. Onu koyacak bir "kutuya" ihtiyacımız vardır.

Neden içine (1, 1) yazıyoruz? (1) desek olmaz mı?

Olmaz, çünkü kuantum bilgisayarları okuma yapıldıktan sonra veriyi saklayamaz.

Kuantum dünyasındaki bilgi uçucudur.

Bu yüzden iki farklı çizgiye (kabloya) ihtiyacımız var

Örnek: Eğer QuantumCircuit(3, 2) yazsaydık:

3 tane Kuantum Çizgimiz (0, 1, 2 indeksli kubitler) ve 2 tane Klasik Çizgimiz (0, 1 indeksli bitler) olurdu.

KAPI (GATE) NEDİR? NEYE GÖRE EKLENİR?

- Kapı nedir?
- Klasik bilgisayarlarda AND, OR, NOT kapıları duymuşsundur. Bunlar elektrik sinyallerini değiştirir. Kuantumda Kapı (Gate) demek, kubitin yönünü, açısını veya durumunu değiştiren komut/lazer darbesi demektir.
- Sen `devre.h(...)` yazdığında, Python devre nesnesinin içindeki `.h()` metodunu (fonksiyonunu) çağırırsın ve "Bu devreye bir Hadamard kapısı perçimle" demiş olursun.
- Neden `devre.h(1)` demedik de `devre.h(0)` dedik?
- Çünkü yazılımda (Python'da) listeler ve indeksler 0'dan başlar.
- Biz devreyi kurarken `QuantumCircuit(1, 1)` dedik. Yani sadece 1 tane kubit istedik.
- Elimizdeki ilk ve tek kubitin adresi (indeksi) 0'dır.
- Eğer `devre.h(1)` yazsaydın, Python sana "Kardeşim sen 1 tane kubit istedin (o da 0 numaralı kubit), olmayan 1 numaralı kubite kapı koyamazsın!" deyip `IndexError` hatası verirdi.
- Eğer devreyi `QuantumCircuit(2, 2)` diye kursaydık, 2. kubite ulaşmak için `devre.h(1)` yazabilirdin.

Hangi Kapılar Var ve Neye Göre Ekliyoruz?

Kapıları yapmak istediğin mantıksal işleme göre seçip eklersin. En çok kullanılan ana kapılar şunlardır:

1. Hadamard Kapısı -> `.h(kubit_no)`

- Ne yapar? Kubiti kesin durumdan çıkartıp %50 0, %50 1 olasılıklı Süperpozisyona sokar.
- Nerede kullanılır? Kuantum bilgisayarının aynı anda bütün ihtimalleri denemesini istediğin an her algoritmanın en başında kullanılır.

2. Pauli-X Kapısı -> `.x(kubit_no)`

- Ne yapar? Klasik NOT (Tersini Alma) kapısıdır. 0 olan kubiti 1, 1 olanı 0 yapar.
- Nerede kullanılır? Bir kubiti başlangıçta 0 durumundan 1 durumuna getirmek istediğinde.

3. CNOT (Controlled-NOT) Kapısı -> `.cx(kontrol_kubiti, hedef_kubiti)`

- Ne yapar? İki kubiti birbirine bağlar (Dolaşıklık / Entanglement yaratır).
- Mantığı: "Eğer kontrol kubiti 1 ise, hedef kubitinin durumunu tersine çevir (0 \ 1 veya 1 \ 0). Eğer kontrol kubiti 0 ise hedef kubite dokunma."
- Nerede kullanılır? Kubitlerin birbiriyle haberleşmesini ve ışınlama (teleportation) gibi kuantum etkileşimlerini sağlamak için.

CNOT (Controlled-NOT) Kapısı Örnek:

```

● ● ●
# Örnek: 2 Kubitli Dolaşıklık Devresi
devre = QuantumCircuit(2, 2)

devre.h(0)
# 0. kubit süperpozisyona sok

devre.cx(0, 1)
# 0. kubit 1 olursa, 1. kubit de 1 yap (İkisini birbirine
# bağladık!)
```

3. Hadamard (H) Kapısını Ekleme

```

● ● ●
# ADIM 3: KUBİTE İŞLEM YAPMA (KAPI EKLEME)
devre.h(0)
```

- $h(\dots)$: Hadamard kapısı demektir. Görevi, içine verilen kubit süperpozisyona sokmaktır. Yani kubit aynı anda hem %50 0 hem %50 1 durumuna getirir (parayı havaya atmak gibi).
- 0: Kaç numaralı kubit bu işlemi uygulayacağımı söyler. Yazılımda saymaya 0'dan başladığımız için ilk kubitimiz 0 indekslidir.

4. Ölçüm Yapma



ADIM 4: ÖLÇÜM (SONUCU BAKMA/KAYDETME)

```
devre.measure(0, 0)
```

Kuantum dünyasından çıkıp klasik dünyaya veri aktardığımız yer burasıdır.

Ölçüm (measure) neden (0,0) girdik?

QuantumCircuit sınıfına ait bir metottur (fonksiyondur).

Kuantum fiziğinde "Ölçüm Problemi" vardır: Bir kubiti ölçmediğin sürece o bir olasılık dalgasıdır. Ama ölçtüğün (baktığın) an çöker ve gerçek bir klasik veri olur.

→ measure (Kubit_İndeksi, Klasik_Bit_İndeksi)
şeklinde çalışır.

→ 0, 0 ifadesi: "0 numaralı kubiti ölç ve çıkan sonucu 0 numaralı klasik bite kaydet." demektir.

Neden İki Tane Parametre Aldı (0, 0)?

Çünkü measure fonksiyonu 2 girdi ister:

measure(ölçülecek_kubit_indeksi,
kaydedilecek_klasik_bit_indeksi)

- Birinci 0: "Git, Kuantum Çizgisindeki 0 numaralı kubiti oku/ölç."
- İkinci 0: "Oradan çıkan sonucu (\$0\$ veya \$1\$), Klasik Çizgideki 0 numaralı bit kutusuna yaz."

Eğer 3 kubitlik bir devren olsaydı şöyle yazabilirdin:



```
devre.measure(2, 1)
# 2 numaralı kubit ölç, sonucu 1 numaralı klasik bite yaz.
```

5. Simülatörde Çalıştırma

Simülatör Nedir?

Gerçek bir kuantum bilgisayarı milyarlarca dolarlık, -273 derece soğutulan dev cihazlardır. Biz kodumuzu test etmek için kendi bilgisayarımızın işlemcisini (CPU) kuantum bilgisayarı taklidi yapmaya zorlarız. İşte bu taklitçi programa Simülatör denir.

Kodda Nasıl Çalışır?:



```
from qiskit_aer import AerSimulator

# 1. Simülatör Nesnesi Oluşturma
simulator = AerSimulator()
```

AerSimulator() bir sınıftır. Bu satırla bilgisayarının belleğinde sanal bir kuantum işlemcisi başlatmış olursun.



```
# 2. Çalıştırma (Run) Fonksiyonu
islem = simulator.run(devre, shots=1000)
```

==> simulator.run(...): Simülatör sınıfının çalıştırma fonksiyonudur (metodudur).

==> devre: Yukarıda çizdiğimiz kapıları ve ölçümü içeren tasarımı veriyoruz.

- shots=1000: Kuantum olayları zar atmak gibidir. Bir zarı 1 kere atarsan "Bu zar hep 6 mı geliyor?" anlayamazsın. shots=1000 diyerek simülatöre: "Bu devreyi baştan sona 1000 kere sıfırlayıp tekrar çalıştır" dersin.
- islem: Bu komut anında sonucu vermez! Arka planda çalışan işin (Job) takip numarasını/objesini döner.



```
# 3. Sonucu Paketten Çıkarma
sonuc = islem.result()
```

islem.result(): biten 1000 adet simülasyonun tüm ham verilerini paket halinde çekip sonuc değişkenine yükler.

Simülatörde Çalıştırma Özet Olarak:



```
# ADIM 5: SİMÜLATÖRÜ HAZIRLAMA VE ÇALIŞTIRMA
similator = AerSimulator()
islem = similator.run(devre, shots=1000)
sonuc = islem.result()
```

- AerSimulator(): Sanal kuantum bilgisayarını çalışmaya hazır hale getirir.
- similator.run(devre, shots=1000): Devremizi simülatöre gönderir.
- shots=1000 parametresi, kuantum deneyini 1000 kez üst üste tekrarlatıyoruz.
- islem.result(): 1000 adet denemenin ham ham verisini paket olarak alır.

6. Sonucu Okuma

Elimizdeki sonuc değişkeni devasa ve karmaşık bir veri paketidir (içinde simülâtörün çalışma süresi, sistem bilgileri vs. de vardır). Bize sadece hangi sonuçtan kaç tane geldiği lazımdır.

a. Metin (Sözlük) Olarak Bakmak: .get_counts()

```
# ADIM 6: SONUÇLARI EKRANA YAZDIRMA  
print(sonuc.get_counts())
```

=> get_counts() fonksiyonu, ham veri paketini tarar ve bize Python Sözlüğü (Dictionary) formatında temiz bir özet verir.

=> get_counts(): "Bana 1000 denemede kaç kere 0, kaç kere 1 geldiğinin özetini/sayımını ver" demektir

```
# çıktı örneği:  
{ '0': 498, '1': 502 }
```

Anlamı: 1000 denemenin 498 tanesinde klasik bite 0 kaydolmuş, 502 tanesinde 1 kaydolmuş.
1000 atışın yaklaşık yarısı 0, yarısı 1 çıkmıştır.

2. Görsel Grafik (Histogram) Olarak Bakmak

İstersen bunu bir çubuk grafiğine dönüştürebilirsin.

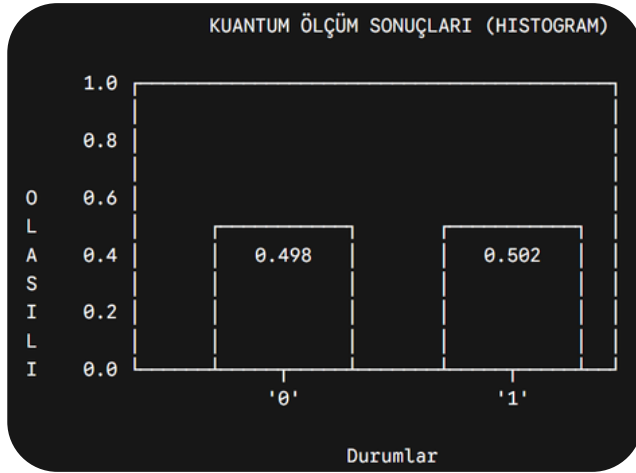
Qiskit'in içinde grafik çizdiren hazır fonksiyonlar vardır:

```
from qiskit.visualization import plot_histogram

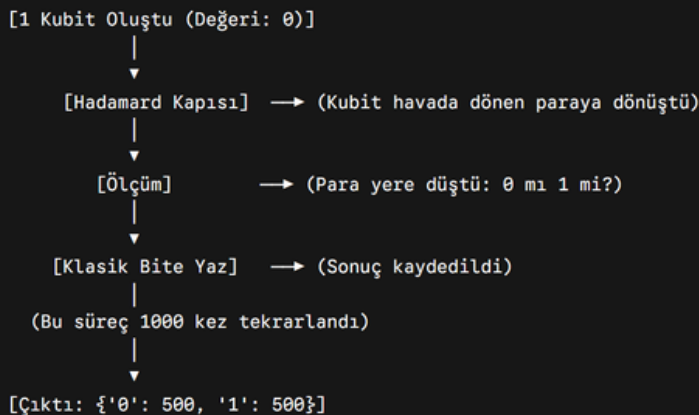
# Sözlük verisini alıp grafik çizer
plot_histogram(sayimlar)
```

Bu fonksiyon sana y ekseninde olasılıkları (0.498 ve 0.502), x ekseninde ise 0 ve 1 durumlarını gösteren görsel bir tablo üretir.

Histogram Çıktısı:



Süreç Nasıl İşliyor? (Görsel Özet)



Kod / Nesne	Türü	Açıklama & Parametreler
QuantumCircuit(n_q, n_c)	Sınıf (Class)	Devre nesnesini türetir. • n_q: Kubit sayısı • n_c: Klasik bit sayısı
devre.h(q)	Metot	Hadamard Kapısı: q indeksli kubitü süperpozisyona sokar.
devre.x(q)	Metot	Pauli-X Kapısı: q indeksli kubitü tersine çevirir (0\1 veya 1\0).
devre.cx(q_kontrol, q_hedef)	Metot	CNOT Kapısı: Kontrol kubitü (q_kontrol) 1 ise, hedef kubitin (q_hedef) değerini tersine çevirir.
devre.measure(q, c)	Metot	q indeksli kubitü ölçer ve sonucu c indeksli klasik bite yazar.
AerSimulator()	Sınıf	Sanal kuantum işlemci ortamını başlatır.
sim.run(devre, shots=N)	Metot	Devreyi N kez simüle eder. Bir Job (İş) nesnesi döndürür.
islem.result()	Metot	Tamamlanan simülasyonun ham veri paketini döndürür.
sonuc.get_counts()	Metot	Ham veriden her durumun kaç kez gerçekleştiğini gösteren bir Python sözlüğü (dict) çıkarır.